

Session 5: Git & Version Control.

Task 1: Essential Git Commands

- **Initialize a repository:** `git init`
- **Stage all files:** `git add .`
- **Commit with a message:** `git commit -m "Your commit message"`
- **Push to remote:** `git push origin main` (or *master*, depending on your branch name)
- **Pull latest changes:** `git pull`

Task 2: git fetch vs. git pull

The Difference:

`git fetch` safely downloads new data and history from a remote repository without altering or merging it into your local working files. Conversely, `git pull` instantly downloads that data **and** automatically merges it into your current local branch right away.

Task 3: Creating and Sharing a GitHub Repo

Because I am an AI, I cannot physically create a live URL hosted on your personal account or browse your local file system to extract a link. However, when you complete this step, your repository link will follow this exact structure:

`https://github.com/your-username/your-repository-name`

Steps to complete this on your system:

1. Go to GitHub and click **New** to create a repository named `my-first-repo`.
2. In your local system terminal, create a text file: `echo "Hello World" > readme.txt`
3. Run these commands to push it:

```
Bash
git init
git add readme.txt
git commit -m "First commit"
git branch -M main
git remote add origin https://github.com/your-username/my-first-repo.git
git push -u origin main
```

4. Copy the URL from your browser address bar and paste it into your assignment.

Task 4: Fork vs. Pull Request

The Difference:

A **Fork** creates an entirely independent, personal copy of someone else's repository under your own GitHub account. A **Pull Request (PR)** is a formal request or proposal you submit to merge changes from your copy back into the original creator's project repository.

Task 5: Branching and Merging Workflow

Here are the step-by-step commands you would use in your terminal to create a branch, commit a change, and merge it back into main:

Bash

1. Create and switch to the new branch

```
git checkout -b feature-test
```

2. (Make your changes to a file here, then stage it)

```
git add .
```

3. Commit the change on the feature-test branch

```
git commit -m "Added temporary test features"
```

4. Switch back to the main branch

```
git checkout main
```

5. Merge the feature-test branch into main

```
git merge feature-test
```

6. (Optional) Delete the test branch now that it is merged

```
git branch -d feature-test
```